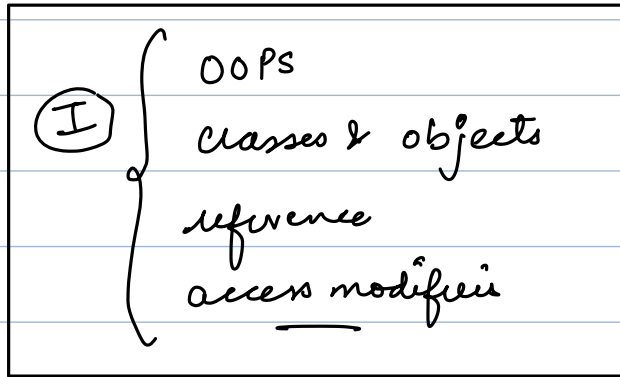


OOPS — ④



②

constructors
deep / shallow
static →

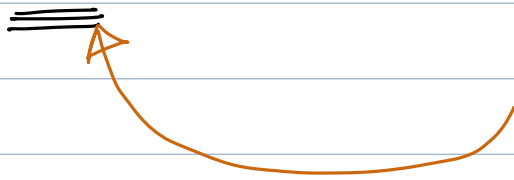
③

Inheritance
Polymorphism

④

Interfaces
Abstract class
Final

OOP



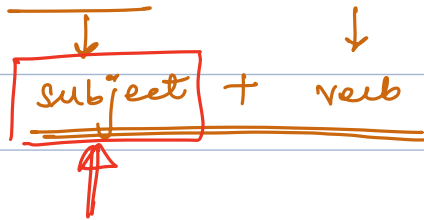
Procedural → C

↓
procedure?

↓
a set of instructions.
fⁿ / method.

fⁿ1 → fⁿ2 → fⁿ3
↑

Mohit is teaching



print Student(rollNo, batch, psp...)
{
=
}

printing is
happening
on student.

st. print ();
↑

student is doing
the printing.

subject/entity has the
control.

Object Oriented Programming

Abstraction
Encapsulation
Inheritance
Polymorphism.

Abstraction

- ↓
- ① just knowing enough necessary details can help you work.

• `sort(____);` comparator] →

• Encapsulation

↓
capsule



→ Class

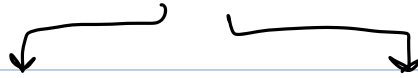
? → ① holds the medicine together

? → ② protects the medicine

OOP



entities



data

behaviours

Student

- id
- name
- roll no
- batch
- psp
- ⋮

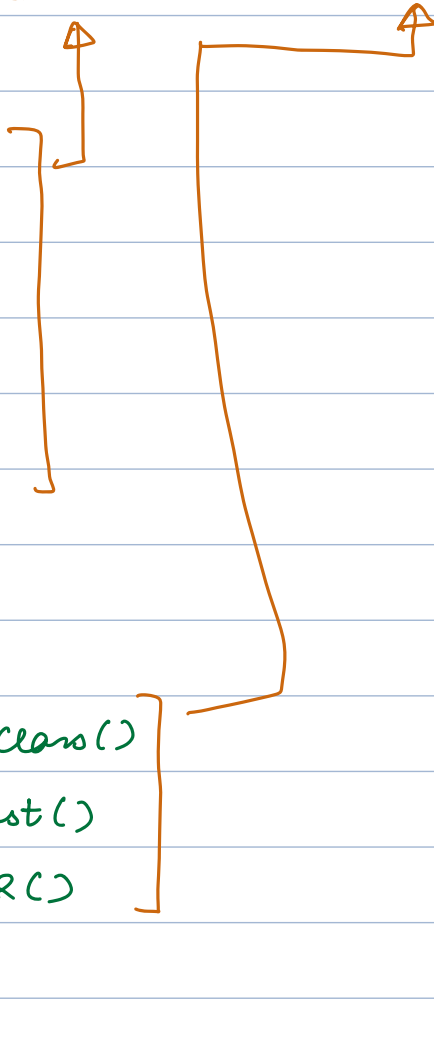
Bird

- #wings
- type
- #legs
- color
- country
- ⋮

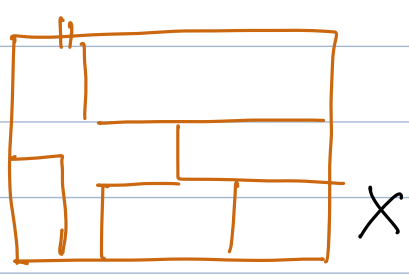
- attendClass()
- rateInst()
- raiseHR()
- ⋮

behaviour

- eat()
- fly()
- makeSound()
- ⋮



blueprint of entity
custom data-type `class Student {`
 `int age;`
 `String name;`
 `String batch;`
 `...`
 `}`



`void attendClass() {`

`=`

`}`

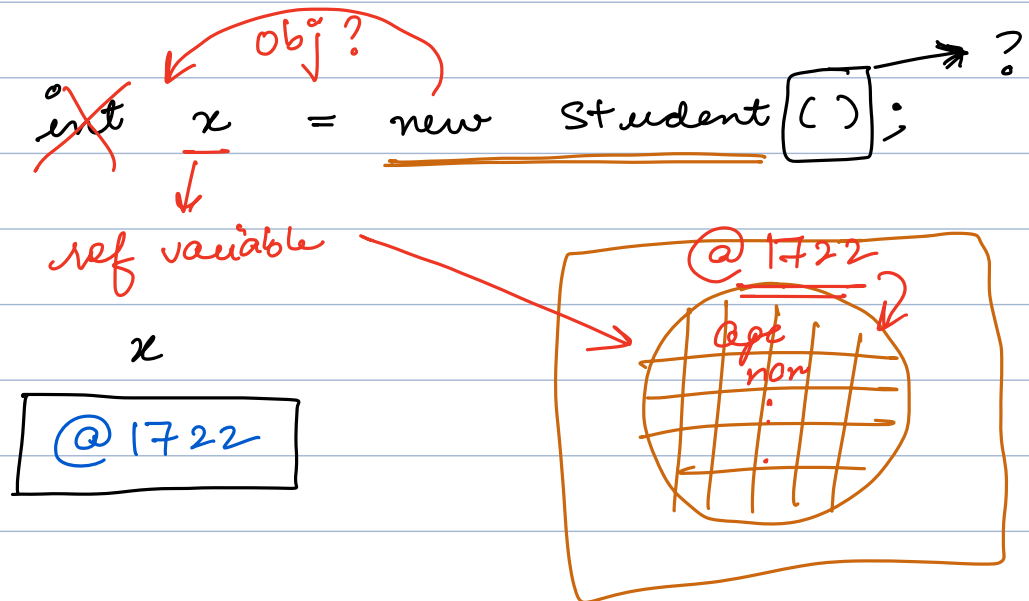
`...`

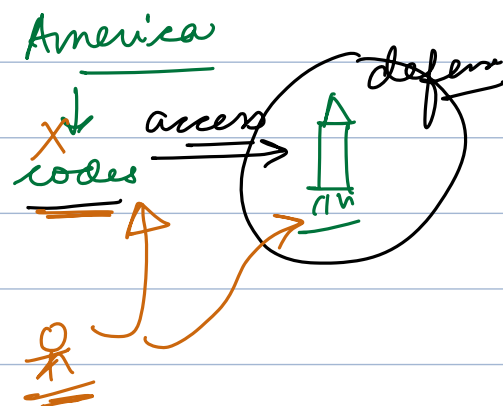
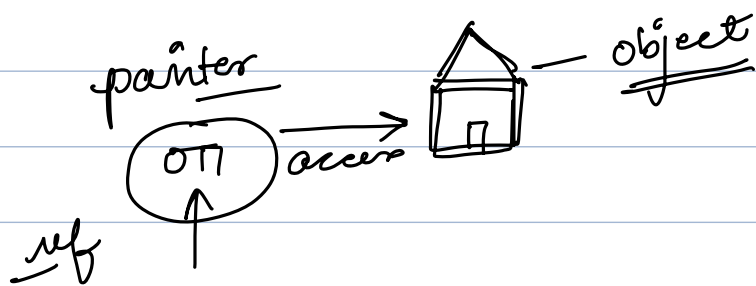
`}`

class $\longrightarrow$ existence $\longrightarrow$ object.

Student ~~int~~ x = new Student();

obj? (pointing to new Student())
ref variable (pointing to x)
x (pointing to box containing @1722)
@1722 (pointing to memory address in object diagram)
age non (pointing to attribute in object diagram)





$x.age = 25;$
 $\downarrow$
 $@1722.age = 25$

Student $x = \text{new student}();$

$x.age = 45;$

Student $y = x;$

$y.age += 10;$
 $@2000.age += 10;$

$\text{print}(x.age);$
 $@2000.age =$

45

55

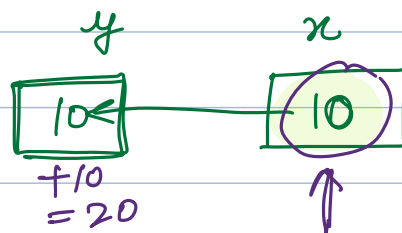
This doesn't create the copy of obj, instead copy of ref.

$\text{int } x = 10;$
 $\text{int } y = x;$

$y += 10;$

$\text{print}(x);$

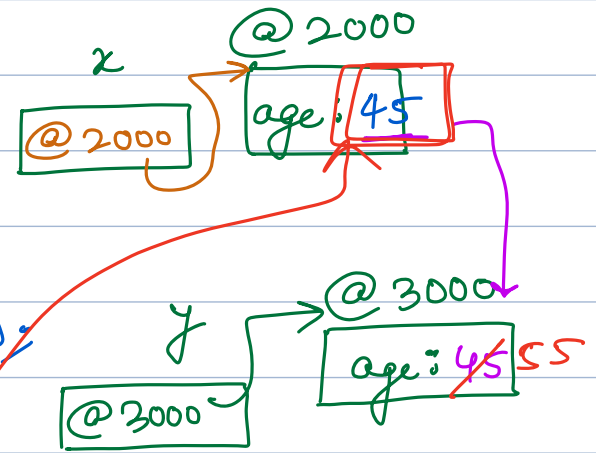
10



```
Student x = new Student();
x.age = 45;
```

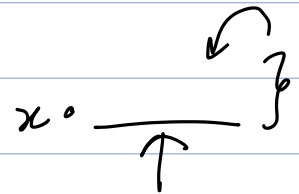
```
Student y = new Student();
y.age = x.age;
y.age += 10;
```

```
print(x.age);
```



$y.age = x.age$
 $@3000.age = @2000.age$

How do I protect?



ACCESS MODIFIERS → data
 → behaviours.

private : within some class
 default : some class + some package
 protected : some class + same package + outside pkg but child class only
 public : anywhere, anyone

folder
 inheritance

→ 10:41

new Student () ;

↓
constructor .

↓
initialise

Java provides one by default if explicitly not provided .
↓
only

↓
access-mod Student () {

}

Student (String name, int age) {

this.name = name;

this.age = age;

⋮

}